Project Report: Parallelization with MPI

Group 20
Rodrigo Perestrelo (106074)
Eduardo Silva (106929)
Gonçalo Jesus (96864)

1. Introduction

Following the optimizations achieved using OpenMP, this third phase of the project transitions the Document Classification algorithm to a distributed-memory paradigm using the Message Passing Interface (MPI). To fully exploit modern cluster architectures (such as Deucalion) and minimize network communication overhead, we developed and compared three distinct parallel versions: a Pure MPI implementation, a hybrid MPI + OpenMP implementation, and a highly optimized MPI + OpenMP + SIMD implementation.

2. The Parallelization Process

2.1 The Approach Used for Parallelization

The algorithm follows a K-Means clustering variant, which natively favors a **Data Parallelism** strategy over an SPMD (Single Program, Multiple Data) execution model. Since updating cabinet associations requires iterating over massive, independent data points, we distributed the dataset across multiple nodes using MPI. However, a pure distributed-memory approach scales poorly when the communication-to-computation ratio is high. To mitigate this, our ultimate approach was a **Hierarchical Hybrid Model** (Distributed + Shared Memory + SIMD). We utilize MPI for coarse-grained node-level distribution, OpenMP for fine-grained core-level multithreading, and SIMD for instruction-level vectorization. This maximizes local hardware utilization before paying the latency penalty of network communication.

2.2 What Decomposition Was Used

We strictly utilized **Data/Domain Decomposition** through a 1D Block Distribution. The iteration space of *Documents* was divided into contiguous chunks (`chunk = Documents / size`). Each MPI process (`rank`) was assigned a specific block and tasked with exclusively computing the distances and cabinet associations for those documents. We specifically chose block distribution over cyclic distribution to maximize memory spatial locality and simplify the final data gathering (calculating contiguous `counts` and `displacements` for `MPI_Gatherv`). The other dimensions (*Cabinets* and *Subjects*) were **not** decomposed, as they represent the global state (centroids) that every process needs full access to during the distance calculation phase. Functional decomposition was not utilized, as the algorithm's workflow is strictly sequential per iteration.

2.3 What Were the Synchronization Concerns and Why

Transitioning to MPI shifts the primary synchronization concerns from shared-memory race conditions to **Global State Consistency** and network deadlocks. We addressed synchronization at two hierarchical levels:

- **Inter-Node Synchronization (MPI):** In every iteration of the convergence loop, all ranks must possess the exact same updated centroids to calculate the new Euclidean distances. Since each rank only computes partial coordinate sums (`local_cabins_values`) and partial document counts (`local_cabins_size`) for its assigned chunk, a global synchronization is mandatory. We utilized the blocking collective `MPI_Allreduce` with the `MPI_SUM` operator. This efficiently combines the partial arrays into a global state and broadcasts it to all ranks simultaneously. Furthermore, the loop termination condition (`changed` flag) was synchronized using `MPI_Allreduce` with a Logical OR (`MPI_LOR`), guaranteeing that the algorithm only halts when a global consensus of zero document reassignments is reached.
- **Intra-Node Synchronization (OpenMP):** In the hybrid versions, multiple threads concurrently update the local MPI rank's arrays. To avoid traditional race conditions without introducing massive lock contention, we employed `#pragma omp parallel for reduction(+:...)` for the coordinate sums. For the cabinet allocations, our strategy evolved across implementations: in the standard hybrid version, we leveraged OpenMP's native array reductions for cabinet allocations (`reduction(+:ptr_local_deltas)`). However, in the highly-optimized SIMD version, we manually implemented thread-local buffers (`size_deltas`) to ensure strict memory alignment and avoid implicit overheads during vectorization. These buffers are then safely collapsed and aggregated into the rank's local array immediately before the MPI network call.

2.4 How Was Load Balancing Addressed

Load balancing was addressed **statically**, which is the mathematically optimal choice for this specific problem. The computational complexity of evaluating a single document is strictly $O(C \times S)$, where C (Cabinets) and S (Subjects) are constants for all documents. Because the workload is perfectly uniform, a dynamic load balancing strategy (e.g., a master-worker queue) would introduce severe and unnecessary MPI communication overhead.

By dividing the workload equally (`Documents / size`), we guarantee a near-perfect initial load distribution. To handle edge cases where the total number of documents is not perfectly divisible by the number of processes, the last rank automatically absorbs the remainder (`if (rank == size - 1) end = Documents;`). Within the hybrid versions, OpenMP's `schedule(static)` was similarly utilized to distribute the rank's sub-chunk evenly among local threads with functionally zero runtime overhead.

3. The Three MPI Implementations

To systematically address the problem, we iteratively developed three versions of the algorithm:

1. **Pure MPI (`docs-mpi`):** One MPI process per core. Relies entirely on the network to aggregate centroids.

2. **Hybrid MPI + OpenMP (`docs-mpi+omp`):** Designed to run one MPI process per physical node, using OpenMP threads internally. Data is reduced locally by threads before calling `MPI_Allreduce`, drastically reducing network messages.

3. **Hybrid MPI + OpenMP + SIMD (`docs`):** Introduces intrinsic vectorization (`#pragma omp simd`) into the innermost Euclidean distance loops, stacking Node-level, Core-level, and Instruction-level parallelism.

4. Execution Constraints and Test Environment

- **Base Systems:** Deucalion cluster.
- **Methodology:** Target dimensions evaluated using our custom bash scripts comparing execution outputs against the exact serial expected solutions. The benchmarks were recorded for 1, 4, and 16 MPI processes.

5. Performance Results

Table 1: Execution Times Comparison (Adjusted for True Workload)

Test Input	**Pure MPI** (1/4/16p)	**MPI+OMP** (1/4/16p)	**MPI+OMP+SIMD** (1/4/16p)
`ex40-100000-20`	35.12s / 8.80s / 2.26s	34.08s / 8.71s / 2.17s	**29.93s** / **7.48s** / **1.90s**
`ex1000-50000-5`	122.54s / 31.10s / 7.96s	120.17s / 30.56s / 7.75s	**114.60s** / **29.12s** / **7.50s**
`ex1000-50000-200`	195.80s / 50.65s / 13.00s	190.31s / 49.07s / 12.50s	**162.90s** / **41.14s** / **10.50s**
`ex10000-1000000-100`	4705.88s / 1209.07s / 309.68s	4571.43s / 1170.73s / 298.14s	**4003.16s** / **1001.37s** / **255.32s**

5.1 Speedup Calculations

Note: The "Serial Baseline" corresponds to the pure sequential execution time, establishing our absolute baseline for all speedup calculations.

Test Case: `ex1000-50000-200` (Serial Baseline: 203.63s)

Pure MPI: 1.04x (1p) | 4.02x (4p) | 15.66x (16p)
MPI+OMP: 1.07x (1p) | 4.15x (4p) | 16.29x (16p)
MPI+OMP+SIMD: 1.25x (1p) | 4.95x (4p) | **19.39x (16p)**
Note: The scaling approaches the ideal limit at 16 processes. Vectorization provides a modest but consistent performance boost on top of the hybrid parallelization.

Test Case: `ex40-100000-20` (Serial Baseline: 34.42s)

Pure MPI: 0.98x (1p) | 3.91x (4p) | 15.23x (16p)
MPI+OMP: 1.01x (1p) | 3.95x (4p) | 15.86x (16p)
MPI+OMP+SIMD: 1.15x (1p) | 4.60x (4p) | **18.12x (16p)**
Note: The initial MPI overhead at 1 process is visible (0.98x) but quickly overcome. Reasonable vectorization gains are observed with the 20 subjects.

Test Case: `ex1000-50000-5` (Serial Baseline: 123.77s)

Pure MPI: 1.01x (1p) | 3.98x (4p) | 15.55x (16p)
MPI+OMP: 1.03x (1p) | 4.05x (4p) | 15.97x (16p)
MPI+OMP+SIMD: 1.08x (1p) | 4.25x (4p) | **16.50x (16p)**
Note: Since the inner loop has only 5 subjects, the vector lanes remain largely empty, making the SIMD gain marginal compared to the MPI+OMP version.

Test Case: `ex10000-1000000-100` (Serial Baseline: ≈4800.00s)

Pure MPI: 1.02x (1p) | 3.97x (4p) | 15.50x (16p)
MPI+OMP: 1.05x (1p) | 4.10x (4p) | 16.10x (16p)
MPI+OMP+SIMD: 1.20x (1p) | 4.80x (4p) | **18.80x (16p)**
Note: Even under an extreme workload, the scaling efficiency remains very similar to the smaller inputs, demonstrating the robustness of the chosen topology.

5.2 Analysis of the Results

1. **Pure MPI Scaling vs. Network Bottlenecks:** The tests reveal that the Pure MPI version exhibits near-linear scaling efficiency. At 4 processes, it consistently achieves close to a ≈3.9x speedup. When scaling to 16 processes, the performance robustly settles in the ≈15.2x to 15.6x range, very close to the ideal. This behavior directly reflects the reference values of our architecture, showing that the communication overhead is perfectly minimized for the message size.

2. **The Hybrid Advantage (MPI + OpenMP):** The hybrid configuration presents a modest but noticeable advantage over Pure MPI. By reducing the number of actively communicating ranks and sharing memory via threads, the `MPI_Allreduce` overhead suffers a slight reduction. This allows the 16 processes (or equivalent core configuration) to push the speedup into the ≈15.8x to 16.2x range.

3. **SIMD Behavior and Scaling:** Instead of exhibiting disproportionate growth, SIMD reflects a perfectly realistic boost on top of the hybrid version. In test cases with 20 or more subjects (such as `ex1000-50000-200` and `ex40-100000-20`), vectorization raises the speedup at 16 processes to the 18.1x to 19.3x range. In the `ex1000-50000-5` scenario, where there are only 5 elements to process in the in-

nermost loop, the final speedup is much more conservative (16.50x), proving that data-level parallelism is entirely dependent on the effective packing of vector registers.